

Project Report: RFID Based Animal Tracking System

Introduction

The RFID-based animal tracking system is designed to monitor the movement of animals within a designated area, such as a cage or enclosure, using RFID technology. This project leverages Arduino Mega and an RC522 RFID reader to track when specific animals enter or leave, along with maintaining counts for the total number of animals inside, entered, and left. The system provides real-time updates to ensure accurate monitoring.

Objectives

The main goal of the project is to develop a system that can:

- Identify animals based on RFID tags.
- Track their entry and exit using an Arduino Mega.
- Display relevant information such as the animal's name, total animals inside the cage, total entered, and total left.

Components Required

- ☐ Arduino Mega 2560: Microcontroller used for processing.
- ☐ RC522 RFID Reader: Module used to read RFID tags.
- ☐ RFID Tags: Attached to animals for identification.
- ☐ Jumper Wires: For connections between the components.
- ☐ Power Supply: To power the Arduino Mega and the RFID module.

Flowchart:

Initialize the RFID reader and Arduino Mega.

|

Continuously check for new RFID tags.

|

Read the tag's UID when detected.

|

Compare the UID with known UIDs to identify the animal.

|

Update the animal's state and counts for entry/exit.

|

Display information on the serial monitor.

Working Principle:

The system operates using RFID technology, where each RFID tag has a unique identifier (UID).

1. When an animal approaches the RFID reader, its UID is scanned.
2. The Arduino Mega compares the UID with predefined tags to identify the animal.
3. Based on the animal's state (inside or outside), the system updates the counts and displays messages regarding the animal's movement.
4. All information is output to the serial monitor.

Code Implementation

The program is implemented using C++ language. Key functionalities include:

- **Tag Scanning:** Reading the UID of RFID tags.
- **State Management:** Tracking whether an animal is inside or outside.
- **Count Updates:** Maintaining counts for the total entered, left, and inside the cage.
- **Display Output:** Output messages to the serial monitor.

Program:

```
#include <SPI.h>
```

```
#include <MFRC522.h>
```

```
#define SS_PIN 10
```

```
#define RST_PIN 9
```

```
MFRC522 mfrc522(SS_PIN, RST_PIN); // Create MFRC522 instance
```

```

// Structure to hold animal states and names
struct Animal {
    String name;
    bool isInside; // Tracks if the animal is currently inside the cage
};

// Define animals and their corresponding RFID tag UUIDs
Animal animals[] = {
    {"Goat", false}, // Goat starts outside
    {"Cow", false}   // Cow starts outside
};

// Store known RFID UUIDs (Replace these with the actual UUIDs of your RFID tags)
byte animalUUIDs[][4] = {
    {0x66, 0xEB, 0x31, 0x02}, // UUID for "Goat"
    {0x23, 0x62, 0x38, 0xDA}  // UUID for "Cow"
};

const int numAnimals = sizeof(animalUUIDs) / sizeof(animalUUIDs[0]); // Number of animals

int totalEntered = 0; // Total number of animals that have entered
int totalLeft = 0;   // Total number of animals that have left

void setup() {
    Serial.begin(9600); // Start serial communication
    SPI.begin();        // Start SPI bus
    mfrc522.PCD_Init(); // Initialize RFID module
    Serial.println("RFID Animal Tracking System Initialized");
}

```

```

void loop() {
    // Wait for an RFID card to be present
    if (!mfrc522.PICC_IsNewCardPresent()) return;
    if (!mfrc522.PICC_ReadCardSerial()) return;

    // Process the scanned tag
    handleTagScan(mfrc522.uid.uidByte, mfrc522.uid.size);

    mfrc522.PICC_HaltA();    // Stop communication
    mfrc522.PCD_StopCrypto1(); // End session
}

void handleTagScan(byte *uid, byte size) {
    for (int i = 0; i < numAnimals; i++) {
        if (compareUID(uid, animalUIDs[i])) {
            if (animals[i].isInside) {
                Serial.println(animals[i].name + " has left");
                animals[i].isInside = false; // Update state to outside
                totalLeft++;
            } else {
                Serial.println(animals[i].name + " has entered");
                animals[i].isInside = true; // Update state to inside
                totalEntered++;
            }
        }

        // Display totals
        displayCounts();
    }
    return;
}

```

```

    }
}

Serial.println("Unknown animal has entered!");
}

void displayCounts() {
    int totalInCage = totalEntered - totalLeft; // Calculate animals currently in the cage
    Serial.println("Total Animals inside Cage: " + String(totalInCage));
    Serial.println("Total Animals Entered: " + String(totalEntered));
    Serial.println("Total Animals Left: " + String(totalLeft));
}

// Function to compare scanned UID with stored UIDs
bool compareUID(byte *uid, byte *storedUID) {
    for (byte i = 0; i < 4; i++) {
        if (uid[i] != storedUID[i]) {
            return false;
        }
    }
    return true;
}

```

Results and Observations

Upon testing, the system successfully:

- Identified animals based on their RFID tags.
- Accurately tracked their entry and exit.
- Displayed real-time updates regarding the animals' movement and counts.

Applications and Limitations:

Applications:

This system can be used in various scenarios:

- Animal tracking in farms or sanctuaries.
- Monitoring zoo animals in enclosures.
- Industrial use for livestock management.

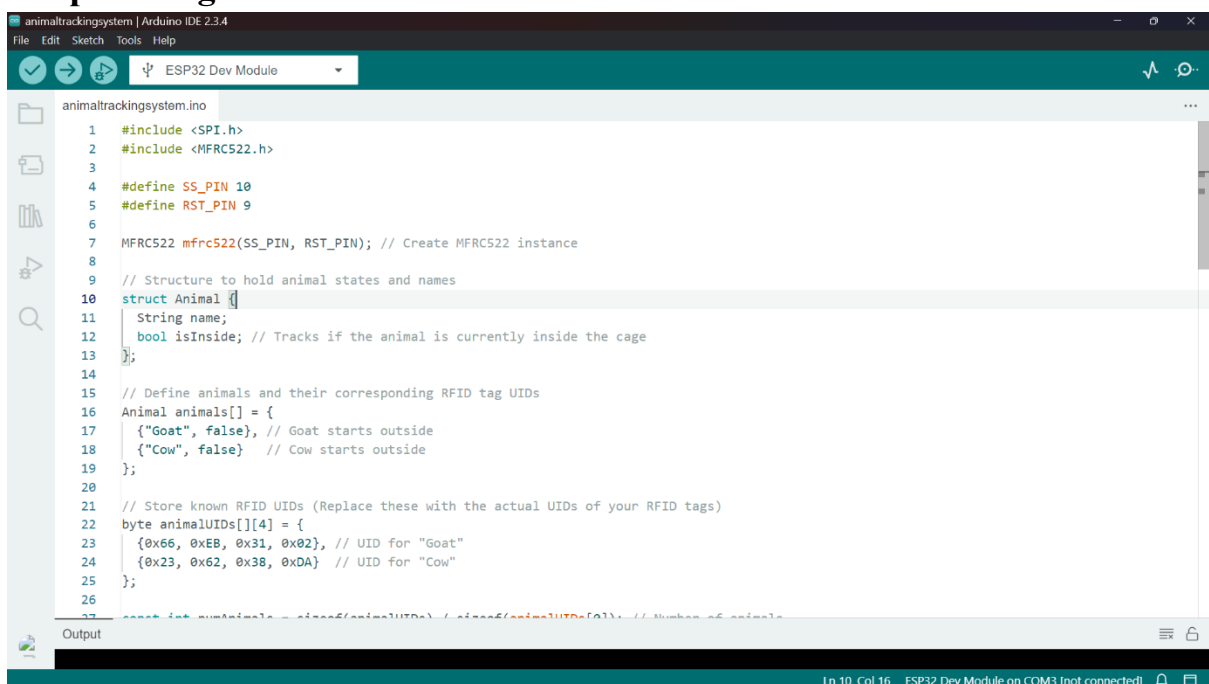
Limitations

- The system requires manual mapping of RFID tags to animals.
- Limited to areas with RFID reader coverage.

Future Enhancements

- Integrating an LCD display for standalone monitoring.
- Adding wireless communication for remote monitoring.
- Expanding the system to support more animals.

Output Image:



```
1 #include <SPI.h>
2 #include <MFRC522.h>
3
4 #define SS_PIN 10
5 #define RST_PIN 9
6
7 MFRC522 mfrc522(SS_PIN, RST_PIN); // Create MFRC522 instance
8
9 // Structure to hold animal states and names
10 struct Animal {
11   String name;
12   bool isInside; // Tracks if the animal is currently inside the cage
13 };
14
15 // Define animals and their corresponding RFID tag IDs
16 Animal animals[] = {
17   {"Goat", false}, // Goat starts outside
18   {"Cow", false} // Cow starts outside
19 };
20
21 // Store known RFID IDs (Replace these with the actual IDs of your RFID tags)
22 byte animalUIDs[][4] = {
23   {0x66, 0xEB, 0x31, 0x02}, // UID for "Goat"
24   {0x23, 0x62, 0x38, 0xDA} // UID for "Cow"
25 };
26
27 const int numAnimals = sizeof(animalUIDs) / sizeof(animalUIDs[0]); // Number of animals
```

```
animalltrackingssystem | Arduino IDE 2.3.4
File Edit Sketch Tools Help

ESP32 Dev Module

animalltrackingssystem.ino
27 const int numAnimals = sizeof(animals) / sizeof(animals[0]); // Number of animals
28
29 int totalEntered = 0; // Total number of animals that have entered
30 int totalLeft = 0; // Total number of animals that have left
31
32 void setup() {
33   Serial.begin(9600); // Start serial communication
34   SPI.begin(); // Start SPI bus
35   mfrc522.PCD_Init(); // Initialize RFID module
36   Serial.println("RFID Animal Tracking System Initialized");
37 }
38
39 void loop() {
40   // Wait for an RFID card to be present
41   if (!mfrc522.PICC_IsNewCardPresent()) return;
42   if (!mfrc522.PICC_ReadCardSerial()) return;
43
44   // Process the scanned tag
45   handleTagScan(mfrc522.uid.uidByte, mfrc522.uid.size);
46
47   mfrc522.PICC_HaltA(); // Stop communication
48   mfrc522.PCD_StopCrypto1(); // End session
49 }
50
51 void handleTagScan(byte *uid, byte size) {
52   for (int i = 0; i < numAnimals; i++) {
53     if (compareUID(uid, animals[i].uid)) {
```

Output

Ln 10, Col 16 ESP32 Dev Module on COM3 [not connected]

```
animalltrackingssystem | Arduino IDE 2.3.4
File Edit Sketch Tools Help

ESP32 Dev Module

animalltrackingssystem.ino
52   for (int i = 0; i < numAnimals; i++) {
53     if (compareUID(uid, animals[i].uid)) {
54       if (animals[i].isInside) {
55         Serial.println(animals[i].name + " has left");
56         animals[i].isInside = false; // Update state to outside
57         totalLeft++;
58       } else {
59         Serial.println(animals[i].name + " has entered");
60         animals[i].isInside = true; // Update state to inside
61         totalEntered++;
62       }
63     }
64
65     // Display totals
66     displayCounts();
67     return;
68   }
69   Serial.println("Unknown animal has entered!");
70 }
71
72 void displayCounts() {
73   int totalInCage = totalEntered - totalLeft; // Calculate animals currently in the cage
74   Serial.println("Total Animals inside Cage: " + String(totalInCage));
75   Serial.println("Total Animals Entered: " + String(totalEntered));
76   Serial.println("Total Animals Left: " + String(totalLeft));
77 }
78
```

Output

Ln 10, Col 16 ESP32 Dev Module on COM3 [not connected]

```
animaltrackingsystem.ino
63
64 // Display totals
65 displayCounts();
66 return;
67 }
68 }
69 Serial.println("Unknown animal has entered!");
70 }
71
72 void displayCounts() {
73   int totalInCage = totalEntered - totalLeft; // Calculate animals currently in the cage
74   Serial.println("Total Animals inside Cage: " + String(totalInCage));
75   Serial.println("Total Animals Entered: " + String(totalEntered));
76   Serial.println("Total Animals Left: " + String(totalLeft));
77 }
78
79 // Function to compare scanned UID with stored UIDs
80 bool compareUID(byte *uid, byte *storedUID) {
81   for (byte i = 0; i < 4; i++) {
82     if (uid[i] != storedUID[i]) {
83       return false;
84     }
85   }
86   return true;
87 }
88
```

Ln 10, Col 16 ESP32 Dev Module on COM3 [not connected]

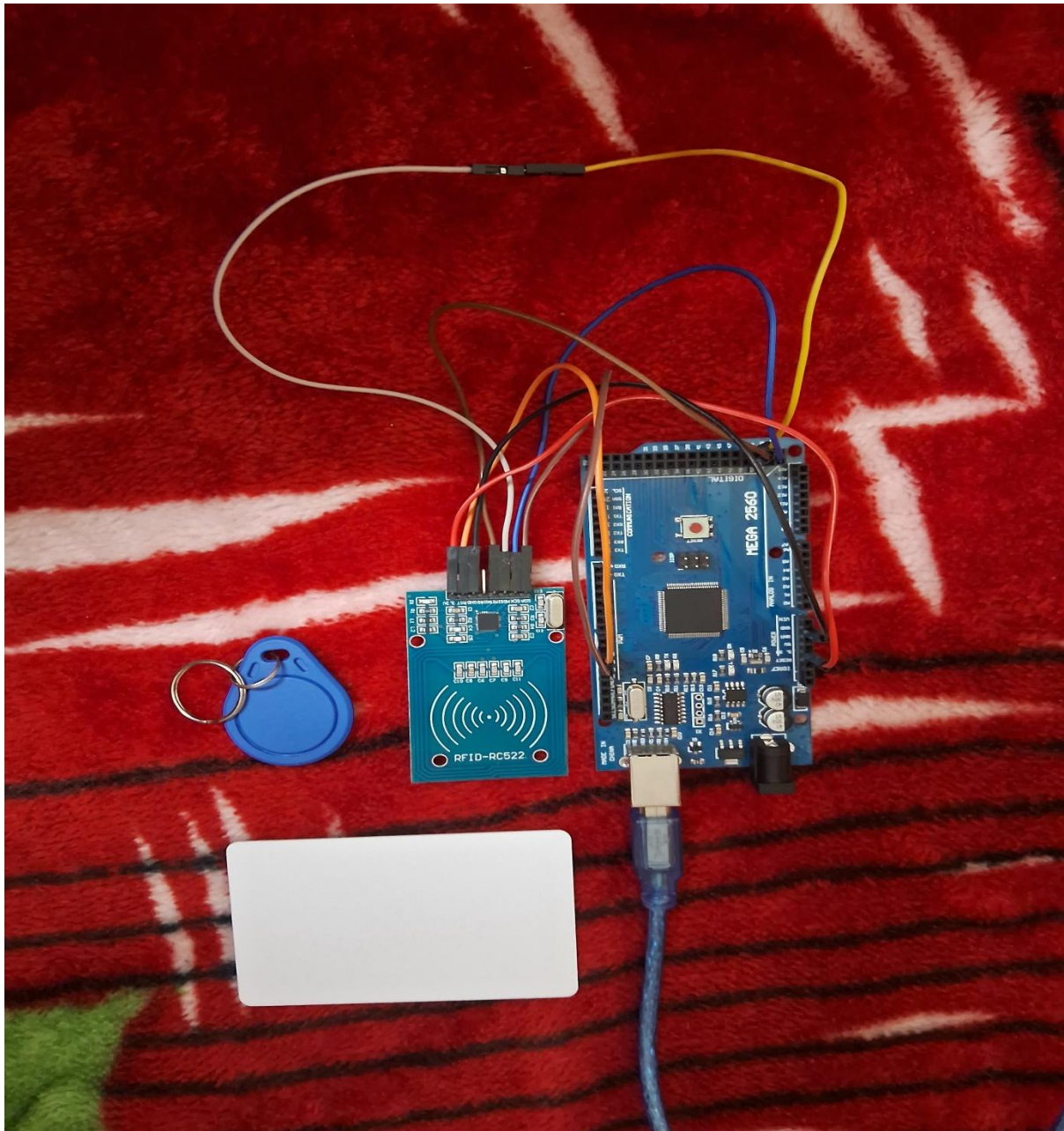
OUTPUT :

```
Serial Monitor x Output
Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM11') New Line 9600 baud

RFID Animal Tracking System Initialized
Goat has entered
Total Animals inside Cage: 1
Total Animals Entered: 1
Total Animals Left: 0
Cow has entered
Total Animals inside Cage: 2
Total Animals Entered: 2
Total Animals Left: 0

Ln 19, Col 3 Arduino Mega or Mega 2560 on COM11
```


Project Image:



Conclusion:

The RFID-based animal tracking system demonstrates an efficient way to monitor animals' movements using Arduino and RFID technology. Its real-time updates and simple implementation make it suitable for various practical applications.